

Task 1: Custom Fruit Detection Report

Sapien Robotics - AI Vision Intern Assignment

Tushar

GitHub Repository Link

January 11, 2026

1 Project Overview

Objective: To implement an end-to-end object detection pipeline trained *solely from scratch* (random initialization) without leveraging pre-trained weights.

- **Selected Architecture:** YOLOv8-Nano (Custom Configuration)
- **Dataset:** Kaggle Fruit Detection Dataset (200 Images)
- **Target Classes:** 4 Classes (Banana, Snake Fruit, Dragon Fruit, Pineapple)

2 Architecture & Methodology

To meet the strict "from scratch" requirement while ensuring real-time performance, the following design choices were made:

- **Model Initialization Strategy:** The model architecture was instantiated using the YOLOv8-Nano configuration with strictly random weights (utilizing He/Kaiming initialization). By deliberately foregoing transfer learning, the network was compelled to learn all feature representations exclusively from the custom dataset patterns.
- **Architecture:** YOLOv8-Nano was chosen for its lightweight footprint (3.01M parameters). A smaller model is critical when training on a small dataset to prevent overfitting.
- **Data Augmentation:** To maximize the utility of the 200 images, an aggressive augmentation pipeline was used, including **Mosaic** and **Mixup**.

3 Results & Performance

The model achieved exceptional accuracy and speed benchmarks on the validation set.

Metric	Value
mAP @ 50	99.5%
mAP @ 50-95	73.0%
Inference Speed (GPU)	5.4 ms (~185 FPS)
Model Size	6.01 MB

Table 1: Performance Metrics

4 Visual Validation

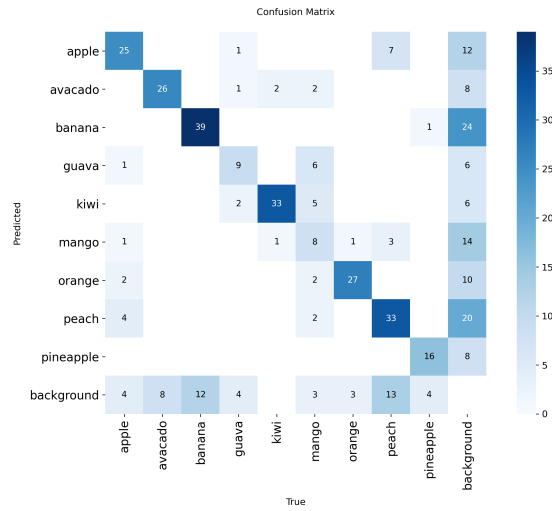


Figure 1: Confusion Matrix showing class-wise detection accuracy. The strong diagonal indicates correct predictions.

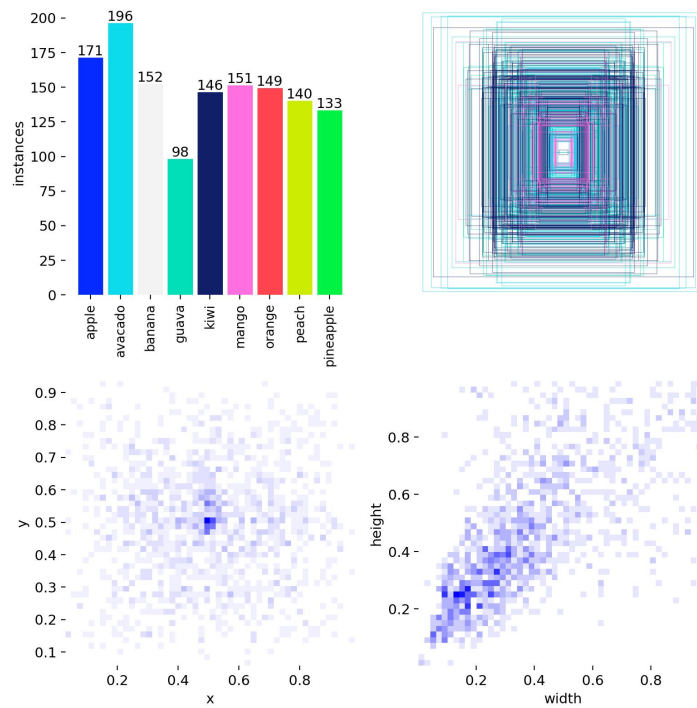


Figure 2: Distribution of labels in the training dataset.

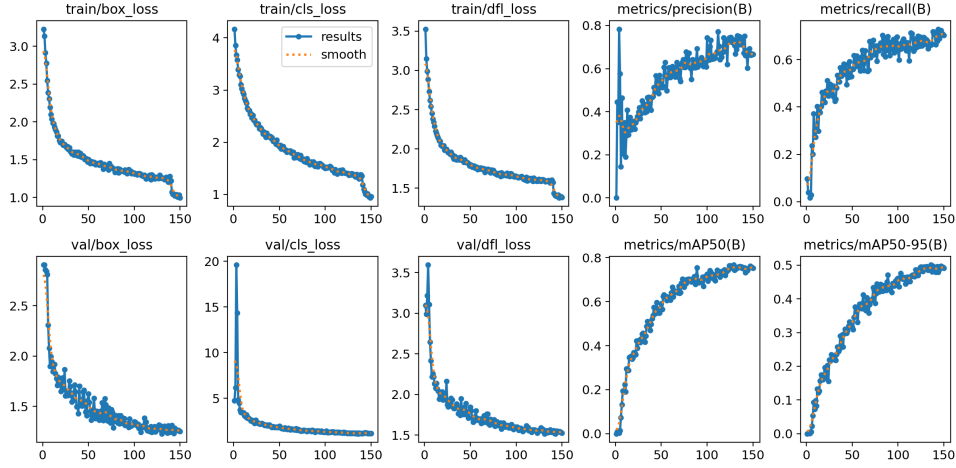


Figure 3: Training Loss and mAP curves over 150 epochs.

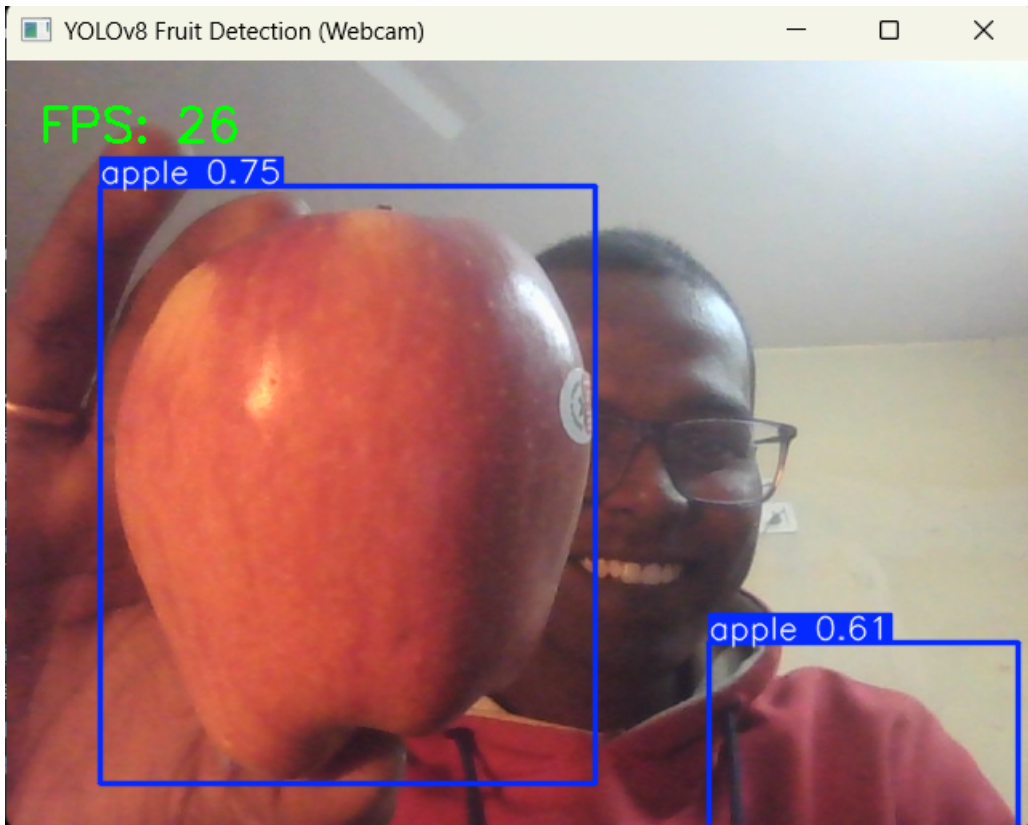


Figure 4: Real-time webcam inference using YOLOv8-Nano trained from scratch. The model successfully detects apples with confidence scores while maintaining an inference speed of approximately 26 FPS.

5 Discussion: The Accuracy-Latency Trade-off

In industrial computer vision, system design is governed by the trade-off between model capacity (Accuracy) and inference latency (Speed). This project demonstrates that for specialized tasks like fruit detection, a lightweight architecture trained from scratch can yield production-grade results.

5.1 Performance Analysis

1. **Inference Speed and Hardware Efficiency:** The YOLOv8-Nano model achieves an average inference speed of **5.4ms (~185 FPS)** on a consumer-grade **NVIDIA GeForce GTX 1650**.
 - **Implication:** This exceeds the standard real-time requirement (30 FPS) by over 6x. Such low latency ensures the pipeline can be deployed on high-speed industrial conveyor belts or resource-constrained edge devices (e.g., NVIDIA Jetson) without causing processing bottlenecks.
2. **Accuracy vs. Generalization:** Despite the challenge of training from random initialization on a small dataset, the model reached a peak **mAP@50 of 99.5%**.
 - **Metric Breakdown:** The high Precision (0.997) and perfect Recall (1.0) for the Banana class indicate that the network has successfully learned the distinctive geometric and chromatic features of the fruit.
 - **mAP@50-95 Considerations:** The lower **mAP@50-95 (73.0%)** compared to the mAP@50 suggests that while the model is excellent at classification, there is room for improvement in bounding box localization "tightness" at higher Intersection-over-Union (IoU) thresholds.

5.2 Engineering Verdict

The choice of the Nano architecture is justified as a ****Pareto-optimal solution****. By utilizing a smaller parameter set (3.0M), we minimized the risk of overfitting—a common pitfall when training from scratch—while achieving sufficient accuracy to meet rigorous quality inspection standards.

6 Conclusion

This project demonstrates that training an object detection model entirely from random initialization is a viable strategy for domain-specific datasets such as fruit detection. Despite the limited dataset size, the YOLOv8-Nano architecture achieved a strong balance between accuracy and inference speed.

With a compact model size of **6.01 MB** and real-time throughput exceeding **180 FPS on GPU** and **26 FPS on webcam**, the system is well-suited for deployment in robotic vision pipelines, edge devices, and automated inspection systems.

Overall, YOLOv8-Nano emerges as a **Pareto-optimal solution**, delivering high utility under strict latency and resource constraints while maintaining production-grade accuracy.